

Using Tree Traversal

If you program in Python and JavaScript, you're used to working with list, array, and dictionary data structures. You'll encounter tree data structures only if you are dealing with low-level details of certain computer science algorithms such as abstract syntax trees, priority queues, Adelson-Velsky-Landis (AVL) trees, and other concepts beyond the scope of this book. However, trees themselves are simple enough concepts.

A *tree data structure* is a data structure composed of nodes that are connected to other nodes by edges. The *nodes* contain data, while the *edges* represent a relationship with another node. Nodes are also called *vertices*. The starting node of a tree is called the *root*, and the nodes at the end are called *leaves*. Trees always have exactly one root.

Parent nodes at the top have edges to zero or more *child nodes* beneath them. Therefore, leaves are the nodes that do not have children, parent nodes are the non-leaf nodes, and child nodes are all the non-root nodes. Nodes in a tree can have edges to multiple child nodes. The parent nodes that connect a child node to the root node are also called the child node's *ancestors*. The child nodes between a parent node and a leaf node are called the parent node's *descendants*. Parent nodes in a tree can have multiple child nodes. But every child node has exactly one parent, except for the root node, which has zero parents. In trees, only one path can exist between any two nodes.

[Figure 4-1](#) shows an example of a tree and three examples of structures that are not trees.